

TDA - Teoría 2. Divide & Conquer

Idea general.

Es una técnica algorítmica y consiste en:

1. Dividir el problema en subproblemas más pequeños del mismo tipo.
2. Conquistar resolviendo recursivamente cada subproblema.
3. Combinar las soluciones de los subproblemas en la solución global.

Esquema General:

$F(x)$:

Si x es suficientemente simple:
resolver en forma directa.

Sino:

dividir x en $x_1, x_2, \dots, x_k$

para cada x_i :

$$y_i = F(x_i)$$

combinar los y_i en y
devolver y .

Análisis de complejidad.

Un algoritmo D&C divide un problema de tamaño n en:

- a subproblemas de tamaño n/c ,
- con costo de combinación $f(n)$,

tiene recurrencia:

$$T(n) = a \cdot T(n/c) + f(n)$$

Teorema Maestro:

Para $T(n) = aT(n/c) + f(n)$:

1. Si $f(n) = O(n^{\log_c a - \epsilon})$
 $\rightarrow T(n) = \Theta(n^{\log_c a})$

2. Si $f(n) = \Theta(n^{\log_c a} \cdot \log^k n)$
 $\rightarrow T(n) = \Theta(n^{\log_c a} \cdot \log^{k+1} n)$

3. Si $f(n) = \Omega(n^{\log_c a + \epsilon})$ y cumple regularidad
 $\rightarrow T(n) = \Theta(f(n))$

Ejemplos clásicos:

1. Búsqueda Binaria:

- Divide: partir el arreglo a la mitad.
- Conquer: buscar en la mitad relevante.
- Combine: trivial (no hay combinación).
- Recurrencia: $T(n) = T(n/2) + O(1)$.
- Complejidad: $O(\log n)$.

2. Merge Sort:

- Divide: 2 mitades.
- Conquer: ordenar cada mitad.
- Combine: merge de tamaño $O(n)$.
- Recurrencia: $T(n) = 2T(n/2) + O(n)$.
- Complejidad: $O(n \log n)$.

3. Subarray de suma Máxima (Maximum Subarray):

- Divide: en 2 mitades.
- Conquer: mejor subarray en izquierda o derecha.
- Combine: mejor subarray q cruce el medio ($O(n)$).
- Recurrencia: $T(n) = 2T(n/2) + O(n)$.
- Complejidad: $O(n \log n)$.

Observaciones importantes:

- No toda recurrencia es D&C:
Ejemplo: búsqueda lineal recursiva divide en 1 y $n-1$, lo cual no es balanceado $\rightarrow$ complejidad sigue siendo $O(n)$, no mejora.
- Para q sea D&C útil, la división debe ser significativa y balanceada.
- Los casos típicos de análisis:
 - $a < c$: pocos subproblemas $\rightarrow$ lineal.
 - $a = c$: costo $O(n \log n)$.
 - $a > c$: costo dominado por cantidad de subproblemas.

TDA - Teórica 2. Backtracking.

1. Definición e idea central

El backtracking es una técnica de diseño de algoritmos que consiste en:

- Explorar el espacio de soluciones de manera sistemática.
- Construir soluciones parciales paso a paso.
- Retroceder (backtrack) cuando se detecta que una rama no puede llevar a una solución válida.

→ Se lo puede ver como una búsqueda con podas:

- Sin poda → fuerza bruta (generate & test).
- Con poda → backtracking.

2. Espacio de soluciones.

- Una solución candidata se representa como un vector $a = (a_1, a_2, \dots, a_n)$ donde cada componente pertenece a un dominio finito y ordenado A_i .

- El espacio de soluciones es el producto cartesiano:

$$A_1 \times A_2 \times A_3 \times \dots \times A_n.$$

- Una solución parcial es un prefijo del vector, y puede o no extenderse a una solución completa.

El proceso se modela naturalmente como un árbol de recursión, donde:

- Cada nodo = solución parcial.
- Hojas = extensiones posibles.
- Podar = no generar ciertos hijos pq no llevan a solución válida.

3. Esquema general del algoritmo.

Pseudocódigo (para todas las soluciones):

↑ $BT(a, k)$:

Si a es solución:
 procesar(a)
 return

Sino:

 para cada a_i en $Sucesores(a, k)$:
 $BT(a_i, k+1)$.

Para una solución: (corta en cuenta encuentra una):

BT(a, t):

Si a es solución:
procesar(a)
return.

Sino:

para cada a_i en $Succesor(a, t)$:
BT(a_i , $t+1$).

4. Tipos de Poda.

• Poda por factibilidad: descarta soluciones parciales y violan restricciones.

Ej: en suma de subconjuntos, si la suma parcial excede $\frac{t}{2}$.

• Poda por optimalidad: descarta soluciones parciales y no pueden mejorar la mejor encontrada.

Ej: en mochila, si aun tomemos todos los elementos restantes no superamos el mejor beneficio actual.

Estos podas son lo y diferencia al backtracking de la pura fuerza bruta.

5. Complejidad.

• En el peor caso: sigue siendo exponencial (ej. $O(2^n)$, $O(n!)$).

• En la práctica: con podas bien diseñadas, el espacio a explorar se reduce muchísimo.

• Es especialmente útil en problemas NP-completos donde no se conocen algoritmos polinomiales.

6. Ejemplos clásicos.

De teoría y práctica:

1. Subset Sum / Suma de subconjuntos.

- Representación binaria (tomar / no tomar un elemento).
- Poda: cortar cuando la suma parcial $> t$.

2. N-Reinas.

- Colocar n reinas en un tablero $n \times n$ y se atacan.
- Poda: unificar en cada paso y no haya conflictos de filas, columnas ni diagonales.

3. Mochila (0-1).

- Se reduce a subset sum pero optimizando el beneficio.
- Poda de factibilidad: no superar capacidad.
- Poda de optimalidad (branch & bound): descartar si aun se sumen todos los beneficios restantes no se supera el mejor actual.

4. Sobre

- Backtracking: altamente efectivo en la práctica (muy experimentado en pro- code).

5. Palabras en código (libre práctica):

- Separar una cadena entera en palabras válidas.
- Poda: cortar lo que no es una palabra.

6. Cuadros mágicos:

- Completar una matriz para q filas, columnas y diagonales sumen lo mismo.
- Poda: verificar sumas parciales de filas/columnas antes de completar.

7. Relaciones con otros temas

- Fuerza bruta: explora todo el espacio sin cortar → menos eficiente.
 - Backtracking: explora, pero poda ramas inviables.
 - Programación Dinámica: surge cuando hay subproblemas repetidos → evita recalcular con memoización.
- Ejemplo: subset sum se puede transformar en DP.

8. Ventajas y desventajas.

Ventajas:

- General y aplicable a muchos problemas combinatorios.
- Con poda bien pensadas, muy eficiente en la práctica.
- Encuentra todas las soluciones (o una si se desea).

Desventajas:

- Complejidad exponencial en el peor caso.
- Requiere un buen diseño de poda para ser útil en problemas grandes.

En resumen:

El backtracking es fuerza bruta inteligente: explora todas las soluciones posibles, pero retrocede (backtrack) tan pronto como detecta que una rama es inviable, usando poda de factibilidad y optimidad. Se aplica a problemas combinatorios, de búsqueda y optimización. Debe ser la técnica base antes de pasar a DP o greedy cuando es posible.

TDA - Teoría 3. Programación Dinámica (Top-Down).

1. Origen y Motivación.

- Creada por Richard Bellman en los 50s.
- Se utiliza para resolver procesos de decisión multietapa.
- Nombre: "dynamic programming" para dar idea de planificación y etapas sucesivas.

2. Idea General.

Similar a Divide & Conquer, pero con una diferencia clave:

- Divide & Conquer: subproblemas son independientes.
- Programación Dinámica (PD): los subproblemas se solapan (se resuelven varias veces en recursión ingenua).

→ La PD evita recalcular guardando resultados intermedios (memorización).

3. Propiedades que justifican PD.

Un problema puede resolverse con PD si cumple:

1. Estructura óptima:

La solución óptima del problema puede construirse a partir de soluciones óptimas de subproblemas.

2. Superposición de subproblemas:

La cantidad de subproblemas distintos es mucho menor q la cantidad de llamados recursivos.

Ej. Fibonacci tiene $O(2^n)$ llamados recursivos, pero solo $O(n)$ estados distintos.

4. Enfoques de PD.

1. Top-Down (con memorización).

- Se define recursivamente el problema.
- Se guarda cada resultado en una estructura (array, matriz, diccionario).
- Si el problema ya se resolvió → se resuelve en $O(1)$.
- Ventaja: más natural, se programa desde la definición matemática.

2. Bottom Up (tabulación).

- Se calculan primero los subproblemas pequeños.
- Se llena iterativamente una tabla hasta llegar al problema original.
- Ventaja: Suele ahorrar stack y, a veces, espacio.

5. Esquema general (Top Down).

Pasos para modelar un problema:

1. Definir la función recursiva $f(\text{parámetros})$:
 - Los parámetros deben contener la mínima información necesaria para describir el estado.
2. Semántica de la función:
 - Explicar en palabras y símbolos el valor devuelto.
3. Casos base:
 - Estados triviales donde la respuesta es inmediata.
4. Recurrencia:
 - Cómo se relaciona f con subproblemas más pequeños.
5. Memorización:
 - Guardar los resultados ya calculados en una estructura de memoria.
6. Complejidad:
 - Tiempo $\approx (\# \text{ estados}) \times (\text{costo por estado})$.
 - Espacio $\approx (\# \text{ estados}) \times (\text{tamaño de resultado por estado})$.

6. Ejemplo clásico: Fibonacci.

• Definición recursiva:

$$\begin{cases} F(i) = 1 & \text{si } i \leq 1 \\ F(i) = F(i-1) + F(i-2) & \text{si } i > 1 \end{cases}$$

• Sin memoria: $O(2^n)$

• Con PD Top Down:

- Guardamos cada $F(i)$ en una tabla $MT[i]$.
- Complejidad $O(n)$ tiempo y $O(n)$ espacio.

7. Ejemplos vistos en clase.

1. Coeficientes Binomiales.

• Recurrencia:

$$\begin{cases} C(n, k) = 1 & \text{si } k=0 \text{ o } k=n \\ C(n, k) = C(n-1, k-1) + C(n-1, k) & \text{si } 0 < k < n \end{cases}$$

• Con memorización $\rightarrow$ evita exponencialidad.

2. Problema del cambio (monedas).

• Definición recursiva:

$$\begin{cases} f(s) = 0 & \text{si } s=0 \\ f(s) = \min_{1 \leq i \leq n} (1 + f(s-a_i)) \\ f(s) = \infty & \text{si no hay solución} \end{cases}$$

• Si demuestra y demuestra la cont. mínima de monedas.

3. Mochela 0/1

- Función: $m(k, D)$ = valor óptimo con primeros k objetos y capacidad D .
- Recurrencia:

$$m(k, D) = 0.$$

Si $k=0$ o $D=0$.

$$m(k, D) = m(k-1, D)$$

Si $p_k > D$.

$$m(k, D) = \max(m(k-1, D), b_k + m(k-1, D - p_k)) \text{ en otro caso}$$

- PD Top Down usa matriz de $(n \times C)$ estados.

4. Astro Trade (ejemplo práctico de la guía).

- Modelo: $mgd(x, d)$ = máxima ganancia si tengo x asteroides al final del día d .
- Estados = $O(n^2)$.
- Backtracking sería $O(3^n)$.
- PD Top Down reduce a $O(n^2)$.

5. Rebecas en la granja (como resolviendo arajas).

- Estados: $(x, y, arMod)$ = máxima cantidad de arajas alcanzable desde (x, y) con resto modulo $k+1 = arMod$.
- Se repasa el estado para no guardar la cantidad exacta de arajas, solo el modulo, reduciendo memoria.

8. Complejidad.

- Tiempo:

$$O(\# \text{estados} \times \text{costo por estado}).$$

- Espacio:

Depende de la memoria usada para memorizar.

$$\approx O(\# \text{estados} \times \text{tamaño de resultado por estado}).$$

9. Ventajas y Desventajas.

- ✓ Evita recomputaciones.
- ✓ Más fácil de implementar desde la definición recursiva.
- ✓ Garantiza optimalidad si hay estructura óptima.
- ! Requiere memoria adicional.
- ! No siempre es trivial identificar los parámetros del estado.
- ! Si el número de estados es grande $\rightarrow$ aún puede ser inviable.

10. Pasos de modelado (checklist).

1. ¿El problema tiene estructura óptima y subproblemas?
2. Definir función recursiva (semántica clara).
3. Determinar parámetros mínimos del estado.
4. Establecer casos base.
5. Escribir recurrencia.
6. Aplicar memorización.
7. Analizar estados y complejidad.

TDA - Teórica 4. Programación Dinámica (Bottom-Up).

1. Idea General.

La programación dinámica (PD) se aplica cuando:

- El problema tiene subestructura óptima: la solución óptima depende de subproblemas óptimos.
- Existe superposición de problemas.

Existe dos enfoques:

- Top-Down (memorización): partir de la versión recursiva y guardar resultados.
- Bottom-Up (tabulación): construir iterativamente una tabla desde los casos base hasta la solución final.

→ En Bottom-Up, el orden de cálculo lo fijamos nosotros: vamos resolviendo primero los subproblemas más chicos y avanzamos hasta el problema original.

2. Esquema Bottom-Up.

1. Definir estado: q parámetros identifican a un subproblema.
2. Definir casos base: estados triviales cuya solución conozcamos directa.
3. Definir transición: cómo se calculan estados grandes a partir de más chicos.
4. Orden de cálculo: determinar cómo llenar la tabla para q las dependencias ya estén calculadas.
5. Respuesta final: suele estar en $dp[n]$, $dp[ENTEROS]$ o similar.

Fórmula de complejidad:

Tiempo = #estados $\times$ costo por estado.

3. Ejemplos clásicos:

3.1. Coeficientes binomiales.

Definición:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}, \quad \binom{n}{0} = \binom{n}{n} = 1.$$

Complejidad: $O(nk)$.

Optimización: basta guardar solo la fila anterior $\rightarrow$ memoria $O(k)$.

3.2 Problemas del cambio (Coin Change).

Queremos dar un monto t con la mínima cantidad de monedas.

Definición:

$$f(s) = \min_{a \in S} (1 + f(s - a)), \quad f(0) = 0.$$

- Construyamos $dp[0 \dots t]$.
- Cada posición depende de valores menores.
- Complejidad: $O(k \cdot t)$, donde k la cantidad de monedas.

3.3 Problema de la mochila (0/1 Knapsack).

Dado capacidad C , objetos con pesos p_i y beneficios b_i .

Estado:

$$m(k, D) = \max \{ m(k-1, D), b_k + m(k-1, D - p_k) \}.$$

- Complejidad: $O(nC)$ → pseudopolinomial.
- Optimización: usar solo dos filas, → espacio $O(C)$.

3.4 Longest Increasing Subsequence (LIS, versión pesadillo).

Ejemplo de práctica: "Mis Buenos Aires Crecido".

Estado:

$f(i)$ = longitud máxima de subsecuencia creciente terminada en el edificio i .

Transición:

$$f(i) = 1 + \max_{j < i, h_j < h_i} f(j).$$

- Complejidad: $O(n^2)$ (puede optimizarse a $O(n \log n)$ con estructuras).

3.5 AstroTrade (comprar/vender asteroides).

Estado:

$myr(a, d)$ = max ganancias al final del día d tengo a asteroides.

Transiciones:

- No hacer nada: $myr(a, d-1)$.
- Comprar: $myr(a-1, d-1) - p_d$.
- Vender: $myr(a+1, d-1) + p_d$.

Se llena una matriz $MYR[a][d]$ con complejidad $O(n^2)$.

Optimización: basta con guardar solo el día anterior → espacio $O(n)$.

4. Optimización de memoria.

Una ventaja de Bottom-Up es q permite reducir espacio $\rightarrow$

- Cada estado depende solo de una cantidad acotada de estados anteriores.
- Ejemplo: Fibonacci $\rightarrow$ solo necesitamos 2 variables $\rightarrow$ espacio $O(1)$.
- Ejemplo: AstroTrade $\rightarrow$ depende solo del día anterior $\rightarrow$ espacio $O(1)$ en lugar de $O(n)$.

5. Reconstrucción de la solución.

El valor óptimo no siempre es suficiente: a menudo queremos el camino/selección concreta.

Estrategias:

1. Guardar "pasos" (decisiones) en otra tabla mientras calculamos.
2. Recorrer la tabla desde el resultado final hasta los casos base, siguiendo q transición fue usada.

Ejemplos:

- Mochola: guardar si el objeto k fue incluido o no.
- Cambio: guardar q moneda dio lugar a la solución.
- LIS: guardar el índice previo elegido.
- AstroTrade: guardar q operación se hizo cada día.

6. Comparación Top-Down vs. Bottom-Up.

Aspecto	Top-Down	Bottom-Up.
Implementación	Basada en recursión + memo.	Iterativa con tabla.
Subproblemas calculados	Solo los necesarios	Siempre todos.
Facilidad	Más natural (recursión).	Requiere pensar en orden.
Memoria	Puede ser más alta (stack + Table optimizable memo).	
Reconstrucción	Menos directa.	Más clara.

TDA - Teoría 5 Greedy.

1. Idea General.

- Un algoritmo greedy construye la solución paso a paso, eligiendo en cada momento la opción "localmente óptima" (la mejor disponible en ese instante), sin preocuparse por las consecuencias futuras.
- Son útiles en problemas de optimización: queremos maximizar o minimizar un cierto criterio.
- Muchas veces son heurísticas: dan soluciones buenas en poco tiempo, aunque no siempre son óptimas.
- En algunos problemas, con la estrategia correcta, se garantiza la solución óptima.

2. Componentes Clave.

Para aplicar Greedy, un problema debe tener:

1. Subestructura óptima: Una solución óptima puede construirse a partir de soluciones óptimas de subproblemas.
2. Propiedad de elección greedy: Siempre existe una solución óptima que comienza con la elección que hace el algoritmo greedy.

Métodos de demostración:

- Greedy stays Ahead: se compara paso a paso con una solución óptima y se prueba que el greedy nunca queda "atrás".
- Intercambio (Exchange Argument): se toma una solución óptima y se la modifica, intercambiando elecciones hasta transformarla en la greedy sin perder optimalidad.

3. Ejemplos Clásicos.

3.1. Problema de la mochila.

- Versión fraccionaria:
 - Ordenar objetos por ratio beneficio/peso.
 - Tomar lo que entre de cada uno, empezando por el mayor ratio.
 - El greedy es óptimo.
- Versión entera (0-1):
 - El greedy no garantiza optimalidad (ejemplo: típico: objeto con ratio alto pero impide meter otro conjunto más productivo).
 - Se resuelve con Programación Dinámica, no un greedy.

3.2. Problema del cambio de monedas

- Estrategia greedy: siempre da la moneda más grande posible, y no exceda el monto restante.
- Ejemplo (monedas 1, 5, 10, 25): dando $69 = 2 \cdot 25 + 2 \cdot 10 + 5 + 1 + 1 + 1 + 1$ 8 monedas. ✓ es óptimo.
- Pero con otras monedas (ej. 1, 5, 12, 25) el greedy puede fallar (ejemplo con 20).
- Correcitud depende de las propiedades de las denominaciones.

3.3. Minimización del tiempo de espera (Scheduling by Shortest Job First).

- Un servidor atiende a clientes con tiempos de servicio $t_1, \dots, t_n$.
- Queremos minimizar el tiempo de espera total.
- Estrategia greedy: ordenar por tiempo de servicio creciente.
- ✓ Es óptimo (demostrable con intercambio).

3.4. Tareas con Deadlines (Job Scheduling).

- Cada tarea i dura t_i unidades y tiene deadline d_i .
- Queremos maximizar la cantidad de tareas que se pueden cumplir a tiempo.
- Greedy: ordenar tareas por ~~su~~ deadline creciente e ir ejecutando las que se pueden.
- Correcitud probada con Greedy Steps Ahead.
- Complejidad: $O(n \log n)$ por el ordenamiento.

3.5. Ruta Eficiente (Paradas de combustible).

- Distancia total M , autonomía E , estaciones en posiciones $x_1, \dots, x_n$.
- Queremos minimizar paradas.
- Estrategia greedy: en cada paso, avanzar hasta la estación más lejana alcanzable.
- Es óptimo (se prueba con intercambio).
- Complejidad: $O(n)$.

3.6. Producto Escalar Máximo

- Dados vectores v y w , queremos permutarlos para maximizar

$$\sum v_i(i) \cdot w_i(i).$$

- Estrategia greedy: ordenar uno ascendente y otro descendente.
- Es óptimo (demostrable por intercambio).

4. Características y Complejidad.

- En general los algoritmos greedy son muy eficientes:
 - Muchas veces $O(n \log n)$ por necesitar un ordenamiento.
 - Algunas incluso $O(n)$ si se puede recorrer en una sola pasada.

• Ventajas:

- Fáciles de implementar.
- Rápidos.

• Desventajas:

- No siempre óptimos.
- Dependen de la estructura del problema.

TDA - Teoría 6. Grafos: Representación y Recorridos

1. Introducción

Un grafo modela relaciones entre entidades (vértices). Permite representar redes físicas (carreteras, electricidad, internet) o abstractas (redes sociales, relaciones entre datos, dependencias, etc).

Un grafo $G = (V, E)$ está formado por:

- V : conjunto de vértices o nodos
- E : conjunto de aristas (pares de vértices y están conectados).

Ejemplo:

Si: $V = \{A, B, C, D\}$ y $E = \{(A, B), (A, C), (B, C), (C, D)\}$, el grafo representa 4 vértices y sus conexiones.

2. Tipos de grafos

■ Grafo simple.

No tiene aristas múltiples ni bucles.
Las aristas son pares no ordenados de vértices distintos.

■ Multigrafo.

Permite múltiples aristas entre dos vértices (ejemplo: varias rutas aéreas entre dos ciudades).

■ Pseudografo.

Permite tanto múltiples aristas como bucles (aristas y vértices a su mismo).

En la materia, salvo aclaración, "grafo" se refiere a grafo simple.

3. Terminología básica:

- Grado de un vértice $d(v)$: número de aristas incidentes a v .
 - Ejemplo: si v está conectado a 3 vértices, $d(v) = 3$.
- Máximo grado: $\Delta(G)$
- Mínimo grado: $\delta(G)$.
- Vecindario de v : $N(v) = \{w \in V : (v, w) \in E\}$.

Teorema del grado:

$$\sum_{v \in V} d(v) = 2|E|$$

Cada arista aporta 2 al total (un extremo en cada vertice).

Corolario: la cantidad de vertices con grado impar es siempre par.

4. Tipos de grafos especiales.

- **Grafo completo (K_n):** todos los vertices son adyacentes entre si.
Cantidad de aristas:

$$m = \frac{n(n-1)}{2}$$

- **Grafo complemento ($\bar{G}$):** mismo conjunto de vertices; tiene las aristas q no estan en G .

5. Caminos, recorridos y ciclos.

- **Recorrido:** secuencia de vertices ($u_0, u_1, \dots, u_n$) donde cada par consecutivo es adyacente.
- **Camino:** recorrido sin repetir vertices.
- **Circuito:** recorrido q empieza y termina en el mismo vertice.
- **Ciclo:** circuito sin repetir vertices intermedios.

Ejemplo:

- $P_1 = v_2 \rightarrow v_3 \rightarrow v_1 \rightarrow v_2 \rightarrow v_4$ → recorrido, no camino.
- $P_2 = v_2 \rightarrow v_2 \rightarrow v_5 \rightarrow v_1$ → camino.
- $P_3 = v_2 \rightarrow v_3 \rightarrow v_5 \rightarrow v_4 \rightarrow v_2$ → ciclo.

6. Distancia y conectividad.

- Longitud de un recorrido = cantidad de aristas.
- **Distancia $d(u, v)$** = longitud del camino más corto entre u y v .
 - Si no existe, $d(u, v) = \infty$
 - **Propiedades:**
 1. $d(u, v) \geq 0$, $d(u, v) = 0 \Leftrightarrow u = v$.
 2. $d(u, v) = d(v, u)$.
 3. $d(u, w) \leq d(u, v) + d(v, w)$ (desigualdad triangular).
- **Grafo conexo:** hay camino entre todo par de vertices.
- **Componente conexa:** subgrafo conexo maximal.

7. Subgrafos

Sea $G = (V, E)$:

- Subgrafo: $H = (V_H, E_H)$ tal q $V_H \subseteq V$ y $E_H \subseteq E \cap (V_H \times V_H)$.
- Subgrafo propio: $H \subset G$ (no igual).
- Subgrafo generador: tiene todas las vertices de G .
- Subgrafo inducido por $V' \subseteq V$: incluye todas las aristas entre los vertices de V' .

8. Grafos bipartitos

Un grafo $G = (V, E)$ es bipartito si los vertices pueden dividirse en dos conjuntos disjuntos V_1 y V_2 tales q todas las aristas unen vertices de conjuntos distintos.

- Bipartito completo $(K_{m,n})$: Cada vertice de V_1 se conecta con todos los de V_2 .
- Teorema: G es bipartito $\Leftrightarrow$ no tiene ciclos de longitud impar.

9. Isomorfismo

Dos grafos $G = (V, E)$ y $G' = (V', E')$ son isomorfos si existe una biyección $f: V \rightarrow V'$ tal q

$$(u, v) \in E \Leftrightarrow (f(u), f(v)) \in E'.$$

Esto significa q son "iguales en estructura".

Propiedades q comparten grafos isomorfos:

- Misma cantidad de vertices y aristas.
- Misma número de vertices por grado.
- Igual cantidad de componentes conexas.

!No hay un criterio sencillo (polinomial) q garantice isomorfismo.

10. Representaciones computacionales

Matriz de Adyacencia

Matriz $A \in \mathbb{N}^{n \times n}$ donde:

$$A_{ij} = \begin{cases} 1 & \text{si } (i, j) \in E \\ 0 & \text{caso contrario} \end{cases}$$

- Simétrica en grafos no dirigidos.
- Complejidad espacial: $\Theta(n^2)$.
- Verificar si existe arista: $O(1)$.
- Obtener vecinos de v : $O(n)$.

Propiedades:

- La suma de la fila i = grado de v_i .
- Los elementos diagonales de A^2 indican el grado de cada vértice.

Coste de Adyacencia.

Estructura: vector de listas.

Cada posición v contiene los vértices adyacentes $N(v)$.

Ventajas:

- Espacio: $O(n+m)$.
- Agregar arista: $O(1)$.
- Recorrer vecinos: $O(d(v))$.
- Consultar si existe arista (u, v) : $O(d(v))$.

Ideal para grafos dispersos.

Grafos implícitos.

No se almacenan los aristas: se define el conjunto de vértices y una función de vecindad.

Usados en:

- Tableros (e.g., caballo de ajedrez).
- Espacios de búsqueda (Backtracking, PD).
- Juegos, laberintos, caminos mínimos son grafos explícitos.

11. Digrafos (grafos dirigidos).

Definición:

$G = (V, E)$ con E subconjunto de pares ordenados (u, v) .
Cada arista tiene dirección $\rightarrow$ llamada arco.

- Grado de salida: $d_{out}(u)$ = cantidad de arcos q salen de u .
- Grado de entrada: $d_{in}(v)$ = cantidad de arcos q llegan a v .
- Grafo subyacente: grafo no dirigido q ignora las direcciones.

Matriz de adyacencia dirigida:

$A[i][j] = 1$ si hay arco de $i \rightarrow j$.

- Suma de fila $i = d_{out}(i)$.
- Suma de columna $j = d_{in}(j)$.

Dirigido fuertemente conexo: existe camino dirigido entre cada par (u, v) en ambas direcciones.

12. Recorridos

▲ Recorridos por BFS (Breadth-First Search).

- Estrategia en anchura (nivel por nivel).
- Usa una cola (FIFO).
- Calcula distancia mínima desde un nodo fuente en grafos no ponderados.
- Complejidad: $O(n+m)$.

▲ Recorridos por DFS (Depth-First Search).

- Estrategia en profundidad (recursiva o con pila).
- Útil para detectar ciclos, componentes, topológico, etc.
- Complejidad: $O(n+m)$.

Ambos se basan en la estructura de vecindades (listas de adyacencia).

13. Propiedades y demostraciones importantes

2. Principio del pelo en grafos.
En todo grafo con al menos dos vértices hay al menos dos con igual grado.

2. Suma de grados

$$\sum d(v) = 2|E|$$

(Demostración inductiva en n).

3. Conexos mínimos

Todo grafo con más de $(n-1)(n-2)/2$ aristas es conexo.

4. Ciclo compartido

Si dos caminos usan v y w , hay un ciclo formado por sus aristas.

5. Ciclo impar

Si existe una combinación de longitud impar y empieza y termina en el mismo vértice, hay un ciclo impar.

6. Articulaciones.

Todo grafo conexo tiene al menos dos vértices y no son de articulación.

14. Ventajas y desventajas de representaciones.

Representación	Espacio	Consultar arista	Verificar	Agregar/eliminar arista	Idempotencia
Matriz adyacencia	$O(n^2)$	$O(1)$	$O(1)$	$O(1)$	Grupos densos
Lista adyacencia	$O(nm)$	$O(d_v)$	$O(1)$	$O(1)/O(d_{u,v})$	Grupos dispersos
Hash de adyacencia	$O(nm)$	$O(1)$ amort.	$O(d_{u,v})$	$O(1)$ amort.	Operaciones dinámicas
Implícito	—	depende	depende	—	Grupos generados en tiempo real.

1. Representaciones del Grafo y su Impacto

- No dirigido vs. dirigido: depende la semántica de "camino" (y de ciclo) y cambia las propiedades de DFS (clasificación de aristas) y de conectividad.
- Ponderado vs. no ponderado: si todos los pesos valen 1 (o no hay pesos), BFS de distancias mínimas; para minimizar ponderados no negativos usa Dijkstra (otro tema).
- Estructura de datos:
 - Lista de Adyacencia: espacio $O(n+m)$, recorridos $O(n+m)$.
 - Matriz de Adyacencia: espacio $O(n^2)$, conveniente si el grafo es denso, pero con matriz es $O(n^2)$.
- Convenciones: numerar $0 \dots n-1$; mantener `visitado[u]`, `padre[u]`, y en DFS, tiempos `tin[u]`, `tout[u]`.

2. Recorridos Fundamentales

2.1 BFS (Breadth-First Search, por niveles).

Idea: explora por niveles desde una fuente s , usando cola.

Propiedad central: en grafos no ponderados, la distancia calculada por BFS es el mínimo número de aristas desde s a cada v .

Pseudocódigo (lista de adyacencia):

text.

BFS(G, s):

para v : `dist[v] = +∞`; `padre[v] = ⊥`; `visitado[v] = falso`
`dist[s] = 0`; `visitado[s] = true`
 $Q = \text{cola}()$; $Q.\text{encolar}(s)$.

mientras Q no vacía:

$u = Q.\text{desencolar}()$

para cada w en $Adj[u]$:

si no `visitado[w]`:

`visitado[w] = true`.

`dist[w] = dist[u] + 1`.

`padre[w] = u`.

$Q.\text{encolar}(w)$.

Árbol BFS: las aristas `padre[w] → w` forman un bosque/árbol de niveles (si el grafo es conexo, un árbol).

Complejidad $O(n+m)$ con listas.

Usos clásicos: distancias mínimas (no ponderado), reconstrucción de camino mínimo (retrocediendo por `padre`), chequeo de bipartitismo (coloreando por niveles).

2.2 DFS (Depth-First Search, por profundidad).

Idea: Usa la pila implícita de la recursión para "bajar" hasta donde se pueda y retroceder.

Registra tiempos $t_{in}[u]$ (entrada) y $t_{out}[u]$ (salida), útiles para:

- Clasificar aristas (en dirigidas): arborales, retroceso (back), avance (forward), cruzadas (cross).
- Detectar ciclos (back-edge implica ciclo).
- Orden topológico con "postorden" en DAGs.

Pseudocódigo básico:

$t_{time} = 0$.

DFS(G):

para v : $visitado[v] = \text{falso}$; $padre[v] = L$.

para v :

si no visitado: DFS-Visitar(v).

DFS-Visitar(u)

$visitado[u] = \text{true}$

$t_{in}[u] = ++t_{time}$.

para w en $Adj[u]$:

si no visitado(w):

$padre[w] = u$.

DFS-Visitar(w):

// otros casos: ya visitado $\rightarrow$ clasificar arista según $t_{in}[u]$ vs $t_{in}[w]$.

$t_{out}[u] = ++t_{time}$.

Complejidad: $O(n+m)$ con listas.

Clasificación (dirigidas):

- Tree: lleva a un nuevo vértice.
- Back: hacia un ancestro $\Rightarrow$ ciclo.
- Forward: a un descendiente no padre.
- Cross: entre subárboles distintos (no son ni ancestros ni descendientes uno de otro).

En no dirigido: solo tree y back (toda arista no-arboral es back a un ancestro).

G tiene un ciclo $\Leftrightarrow$ DFS(G) tiene un back edge.
(dirigido o no dirigido).

3. Aplicaciones de BFS/DFS.

3.1 Componentes

- No dirigido: con BFS/DFS desde cada nodo no visitado; cada llamada descubre una componente conexa.
- Dirigido: componentes fuertemente conexas (SCCs) requieren algoritmos específicos (Kosaraju/Tarjan), pero DFS ya aporta estructura (postorden, etc).

3.2 Reconstrucción de caminos y distancias (no ponderados).

- Con ~~DFS~~ de BFS, reconstruís el mínimo camino $s \rightarrow t$: empezá en t y retracedé.
- Distancias: $dist[u]$ en BFS es igual a cantidad de aristas.

3.3 Bipartitos y ciclos

- Bipartito $\Leftrightarrow$ no tiene ciclos impares.
- Algoritmo: colorear con dos colores por BFS/DFS (asignar al vecino el color opuesto).
 - Si encontramos una arista $u-v$ con $color[u] == color[v] \Rightarrow$ no bipartito (ciclo impar).
- En no dirigido, DFS detecta ciclos si aparece una arista a un vértice ya visitado (o back).

3.4 Orden topológico (DAG)

- Kahn (BFS por in-degree):
 - Calcular $in[u]$; encolar los de $in=0$; sacar/visitar, decrementar grados cuando alguno quede en 0, encolar.
 - Si visitaste n vértices $\Rightarrow$ había ciclo.
- DFS: girar en postorden; el orden inverso de salida es topológico.

3.5 Puentes y puntos de articulación (no dirigidos).

- Puente: arista cuya remoción aumenta #componentes.
- Articulación: vértice cuya remoción aumenta #componentes.
- Tarjan/Lowlink (DFS con tin y low):
 - $low[u] = \min(tin[u], \min(low[hijos DFS], \min(tin[w] \text{ de back-edge } u \rightarrow w)))$.
 - Puente $u-v$ si $low[v] > tin[u]$.
 - Articulación:
 - La raíz DFS es articulación si tiene ≥ 2 hijos DFS;
 - para un no-raíz u , si existe hijo v con $low[v] \geq tin[u]$.

4. Árboles y bosques

- Árbol: grafo no dirigido conexo y acíclico; equivalente, $n-1$ aristas; hay un único camino simple entre dos vértices.
- Bosque: conjunto de árboles (acíclicos, no necesariamente conexos).
- Árbol de expansión (spanning tree): subgrafo acíclico q conecta todos los vértices; $n-1$ aristas y es un subconjunto de las aristas originales.
- Árbol BFS/DFS: los pasos inducen un bosque; si el grafo es conexo (no dirigido), se obtiene un único árbol.

5. Árbol Generador Mínimo (AGM/MST).

5.1 Definiciones y propiedades

- Dado un grafo no dirigido y conectado con pesos en aristas, un AGM es un árbol de expansión de peso total mínimo.
- Si el grafo no es conexo, se habla de bosque generador mínimo (uno por componente).
- Propiedades fundamentales:
 - Propiedad de corte (Cut Property): para cualquier corte $(S, V \setminus S)$, una arista de peso mínimo q lo cruza es segura (puede pertenecer a algún AGM).
 - Propiedad del ciclo (Cycle Property): en todo ciclo, la arista de peso máximo no pertenece a ningún AGM (si es única).

Unicidad: si todos los pesos son distintos, el AGM es único.

5.2 Kruskal.

Idea: "agregar aristas baratas sin formar ciclos", en orden creciente de peso.

Estructura: Union-Find (DSU) con compresión de caminos y union por rango.

Pseudocódigo

Kruskal(G):

Sort(E) por peso ascendente.

UF.make-set(v) para todo v .

$T \leftarrow \emptyset$

Para cada arista (u, v, w) en E (en orden):

Si UF.find(u) $\neq$ UF.find(v): // no forma ciclo.

$T \leftarrow T \cup \{(u, v, w)\}$.

UF.union(u, v).

Devolver T .

Correctitud: por propiedad del corte (la seg. arista mínima q conecta dos componentes es segura) y q evitar ciclos respeta propiedad del ciclo.

Complejidad: ordenar $O(m \log n)$; uniones / búsquedas caso $O(1)$ amortizado
 $\Rightarrow O(m \log n)$ (o $O(m \log n)$).

Casos borde: si el grafo no es conexo $\rightarrow$ devuelve un bosque no m.

3.3 Prim

Idea: "cruce" un unico arbol desde un vertice, siempre agregando la arista minima y une el arbol con el resto (un corte dinámico).

Estructura:

- Con cola de prioridades (min-heap) y listas de adyacencia: $O(m \log n)$
- Con matriz (y sin heap): $O(n^2)$ (aplicable en grafos densos o cuando n es moderado).

Pseudocódigo (heap):

Prim(G, s):

para v : $key[v] = +\infty$; $padre[v] = \perp$; $inMST[v] = false$;
 $key[s] = 0$.

$H =$ min-heap de pares $(key[v], v)$.

mientras H no vacio:

$(k, u) = H.pop-min()$

// u queda fijo en el MST

$inMST[u] = verdadero$

para cada $(u, w, peso(u, w))$:

si no $inMST[w]$ y $peso(u, w) < key[w]$:

$key[w] = peso(u, w)$

$padre[w] = u$

$H.decrease-key(w, key[w])$

$T = \{(padre[v], v) : v \neq s\}$

devolver T .

Correctitud: por Propiedad del corte: el corte es (MST parcial, resto), y
semanas siempre la arista minima que une (según)

Complejidad: $O(m \log n)$ con heap (aristas); $O(m + n \log n)$ con
Fibonacci; $O(n^2)$ con matriz.

3.4 Notas prácticas, unicidad, empates y robustez

- Empates: si hay pesos repetidos, puede haber múltiples AGMs; ambos algoritmos siguen siendo correctos.
- Negativos: AGM acepta pesos negativos (no hay problema).
- Descartados: Kruskal entrega bosque; Prim necesita eliminar componente o desconectar "desde cualquier" y reiniciar.
- Reconstrucción: con $padre[v]$ de Prim/Kruskal puedes listar aristas del AGM y sumar su peso.

Definiciones: Camino Minimax y Maximin.

Camino Minimax (también "bottleneck maximizado").

- Después el costo del camino como su peor arista:

$$\text{Costo}_{\min}(P) = \max_{e \in P} w(e).$$

- Un camino minimax entre s y t es el p minimiza ese valor:

$$P^* = \arg \min_{P: s \rightarrow t} \max_{e \in P} w(e).$$

Intuición: quieres el "peor tramo" sea lo más chico posible

Camino Maximin (también "widest path" solo los pesos son anchos de banda)

- Después el valor del camino como su ancho de botella (la arista mínima):

$$\text{Val}_{\min}(P) = \min_{e \in P} w(e).$$

- Un camino maximin entre s y t es el p maximiza ese valor:

$$P^* = \arg \max_{P: s \rightarrow t} \min_{e \in P} w(e).$$

Intuición: quieres el tramo más ancho sea lo más grande posible.

¡Ojo con los nombres: "minimax" = minimizar el máximo;
"maximin" = maximizar el mínimo.

0. Objetivo y alcance

Este apunte integra teoría y práctica con los conocimientos necesarios para resolver la práctica 3.

- Uno a todos: Dijkstra y Bellman-Ford.
- Todos a todos (APSP): Floyd-Warshall, Dantzig, (y programa de Johnson).
- DAGs: DP top-down/bottom-up con orden topológico.
- Técnicas de modelado (SRD / restricciones de diferencias) y "trucos" de casaca (modo fantasma, error DAG de caminos mínimos, etc).

1. Problema de camino mínimo

Dado un grafo abierto (o no dirigido modelado como dirigido) con función de costo $c: E \rightarrow \mathbb{R}$, y un vertice fuente s , buscamos para cada v la mínima suma de costos de un recorrido $s \rightarrow v$ (si existe). Esta formulación es la base de Bellman-Ford y Dijkstra en "Recorrido mínimo uno a todos" de la práctica. Se aplica al problema y la complejidad típica de BF $O(|V||E|)$.

Costos negativos: Si existe un ciclo de costo total negativo alcanzable desde s , el problema "no está bien definido" (las distancias pueden ser $-\infty$). Bellman-Ford detecta su presencia.

2. Dijkstra (no admite costos negativos).

2.1. Hipótesis

Requiere pesos no negativos. Con esta condición, el algoritmo explora el conjunto S de vertices "fijos" es correcto (lema y lema de la teoría).

2.2. Idea e Invariante.

- Mantiene una estimación $\pi(v)$ de la distancia s .
- En cada iteración fija el vertice w_k con menor π fuera de S . $\pi(w_k)$ ya es óptimo.
- Invariante lucroso informal: al finalizar la iteración k , las distancias a todos los vertices de S_k son óptimas.

2.3. Complejidad.

- Con cada probabilidad binaria sobre listas de adyacencia: $O(n \log n)$.
- Impl. naïves es densa puede verse como $O(n^2)$. En las prácticas se usa esta lista al combinar con otras podas.

2.4. Reconstrucción de camino

Guarda predecesores al relajar. Luego reconstruye desde el destino hacia la fuente.

3. Bellman-Ford (admite negativos, detecta ciclos negativos)

3.1 Hipótesis

Permite pesos negativos (no ciclos negativos alcanzables). Si se requieren distancias positivas. Detecta ciclo negativo si en la iteración n aún se puede relajar.

3.2 Idea e Implementación

- Requiere $n-1$ rondas de relajaciones por aristas. Tras k rondas, cada $d(u)$ es la mejor distancia usando a lo sumo k aristas.
- Una ronda adicional verifica si aún mejora: de ser así, hay ciclo negativo.

3.3 Complejidad

Sobre lista de adyacencias, $O(|V||E|)$.

4. Uno-a-todos: técnicas de la práctica

Las clases prácticas sugieren técnicas recurrentes al modelar o post-procesar caminos mínimos.

- Correr desde ambos extremos (fuente y destino en el tras-paso) para filtrar aristas pertinentes a mínimos.
- Armar el "DAG de caminos mínimos" entre dos nodos: conservar solo aristas (u, v) que satisfacen $\text{dist}_S(u) + w(u, v) + \text{dist}_T(v) = \text{dist}_S(t)$. Luego se puede, por ejemplo, correr BFS allí para hallar el camino mínimo con menos aristas entre los optimos por peso.
- Travesía: nodo fantasma para incorporar múltiples fuentes, multiplicar por -1 si se pide máximo, grafo "en niveles" para modelar estados.

5. Caminos mínimos en DAGs

Si el grafo es acíclico dirigido (DAG), el problema se resuelve con DP.

5.1 Formulación recursiva y DP top-down

Si todo vértice es alcanzable desde v y el grafo es DAG, $d(u) = \min_{z \in N^-(u)} \{d(z) + e(z, u)\}$ con base $d(v) = 0$ y memoización (no hay ciclos $\rightarrow$ buen orden).

5.2 DP bottom-up con orden topológico

- Ordenar $v = u_1, \dots, u_n$ con $u_i \rightarrow u_j \Rightarrow i < j$ y relajar en orden: cada arista (u_i, u_j) con $i < j$.

5.3. Tipos prácticos.

Estos atenta a si el grafo del modelo es un DAG: muchas soluciones
bajas de $O(nm)$ a $O(n+m)/O(m)$

6. Todos a todos (APSP): Floyd-Warshall, Dantzig y por encima.
En APSP buscamos distancias $dist(i, j)$ para todos los pares.
Tamaño mínimo del output: $O(n^2)$.

6.1 Complejidad y familia de algoritmos.

- Floyd-Warshall: $O(n^3)$.
- Dantzig: $O(n^3)$.
- series uno a todos:
 - con Dijkstra: $O(n \cdot m \log n)$.
 - con Bellman-Ford: $O(n \cdot m) = O(n^2 m)$.
- Johnson (reordenación + Dijkstra): $O(nm + m \log n) = O(nm \log n)$
(pensamiento de la práctica)

6.2 Floyd-Warshall: invariante y correctitud.

Invariante: tras la iteración k , l_{ij} es la distancia mínima de u_i a u_j usando solo nodos intermedios en $U_k = \{u_1, \dots, u_k\}$ (o sea, k no hay ciclo negativo dentro de U_k). Con eso se prueba el teorema de correctitud.

Preguntas típicas con Floyd: complejidad $O(n^3)$; memoria $O(n^2)$,
cómo recuperar caminos (matriz de "siguientes"), y cómo detectar
ciclos negativos (ver diagonal $l_{ii} < 0$).

7. Modelado por sistemas de restricciones de diferencias (SRD)

Muchos enunciados se reducen a variables x_u y desigualdades del tipo
 $x_u - x_v \leq w(u, v)$. Esto se resuelve como camino mínimo agregando un
nodo fantasma y corriendo BF/Floyd, según el tamaño.

Mini-maze mental:

- "Modelos con aristas $(u \rightarrow v)$ de peso w para cada restricción
 $x_u \leq x_v + w$ "
- "Agregar fuente fantasma s con aristas $(s \rightarrow u)$ de peso 0".
- "Si hay ciclo negativo, el sistema es inconsistente".

8. "DAG de caminos mínimos" y versiones pedidas en la gva.

- DAG de caminos mínimos entre s y t : quedarse con aristas (u,v) y satisfacer $\text{dist}_s(u) + w(u,v) = \text{dist}_s(t)$.
- Permite, por ejemplo, encontrar entre los optimos por peso el de menos aristas por BFS.

9. Resumen de cuando usar cada algoritmo.

- Solo no negativos y uno-a-todos: Dijkstra (mejor lo mas grande: $O(m \log n)$).
- Hay negativos (sin ciclos negativos) o quieres detectar ciclo negativo: Bellman-Ford.
- DAG: DP top-down o bottom-up con orden topológico.
- APSP:
 - grafos densos o no muy grande: Floyd/Dijkstra ($O(n^3)$).
 - grafos grandes y raros con no negativos: Johnson (programa $O(nm \log n)$).
 - o correr a veces uno-a-todos (Dijkstra/BF) según pesos.

10. Checklist de complejidades y condiciones.

- BFS: aristas con costo igual $\rightarrow O(n+m)$.
- Dijkstra: $c(e) \geq 0 \rightarrow O(m \log n)$ (raro) o $O(n^2)$ (denso).
- Bellman-Ford: negativos permitidos; detecta ciclo negativo; $O(nm)$.
- DAG-DP: $O(n+m)$.
- Floyd/Dijkstra: $O(n^3)$ (denso, apsp).
- APSP por JSJP: Dijkstra a veces $O(nm \log n)$ (buena en grafos raros).

11. Checklist para la practica 5 (q saber hacer).

1. Modelar: llevar enunciados a grafos/SRD (agregar nodo fantasma si corresponde).
2. Elegir algoritmo (Dijkstra/BF/DAG/APSP) según pesos y estructura.
3. Implementar con buenas E/S y reconstrucción de caminos (puedes o matrices "siguiente").
4. Detectar ciclo negativo cuando aplique (BF/Floyd).
5. Postprueba: DAG de caminos mínimos, contar caminos en DAG (PD de Cantor), elegir "el más corto entre los mínimos por peso" con BFS.

0. Checklist rápido.

1. Identificar si es un problema de flujo (hay "algo" q se mueve por un grafo con límites/limitaciones).
2. Modelar: ¿Cuáles son los vertices? ¿Que representa cada arista? ¿Capacidades? ¿Unidades por elemento (1) o capacidad general?
3. Fuente (Sumidero) ¿hay varios? Si es, crear super-source y super-sink.
4. Capacidades en vertices: dividir en $v_{in} \rightarrow v_{out}$ con capacidad en la arista intermedia.
5. Cotas inferiores es/demandas: normalizar con corrientes (baja capacidades restando cotas y balancear nodos), y luego max-flow.
6. Matching bipartito, problemas disjuntos: usar modelo clásico de flujo y capacidades.
7. Elegir algoritmo:
 - Rapido y seguro: Edmonds-Karp.
 - Instancias más grandes: Dinic.
8. Comprobar: valor de flujo = capacidad de algún corte $s-t$; obtener el min-cut desde alcantallados en residual.
9. Reconstruir solución del enunciado (emparejamiento, selección de aristas, etc.) leyendo los aristas con flujo > 0 en el modelo.

1. Modelo: Red de Flujo.

Una red de flujo es un grafo dirigido $G=(V,E)$ con:

- Capacidad $c(u,v) \geq 0$ en cada arista $(u,v) \in E$,
- Fuente $s \in V$ y sumidero $t \in V$, $s \neq t$.

Un flujo es una asignación $f: E \rightarrow \mathbb{R}_{\geq 0}$ tal q:

1. (Capacidad) $0 \leq f(u,v) \leq c(u,v)$ para todo $(u,v) \in E$.
2. (Conservación) $\sum_{w:(u,w) \in E} f(u,w) = \sum_{w:(w,u) \in E} f(w,u)$ para todo $u \in V \setminus \{s,t\}$.

El valor del flujo es $|f| = \sum_{(s,u) \in E} f(s,u) - \sum_{(u,s) \in E} f(u,s)$. (Es igual al flujo q sale de s , o entra a t).

Observación (aristas reversas): representaremos aristas reversas en el grafo residual (ver §3). No hace falta guardar $f(v,u) = -f(u,v)$ basta con manejar residual.

2. Cortes $s-t$.

Un corte $s-t$ es una partición (S, T) de V con $s \in S, t \in T$. Su capacidad es

$$\text{cap}(S, T) = \sum_{u \in S, v \in T} c(u, v).$$

Cotas: para cualquier flujo f y cualquier corte (S, T) : $|f| \leq \text{cap}(S, T)$.

3. Grafo residual y caminos aumentantes.

Dado un flujo f , el grafo residual G_f tiene las mismas aristas (y puede incluir reversas) con capacidad residual

- $c_f(u, v) = c(u, v) - f(u, v)$ si $(u, v) \in E$ (avante),
- $c_f(u, v) = f(u, v)$ (retroceder), o $c_f(u, v) \notin E$ original.

Un camino aumentante es un camino $s \rightarrow t$ en G_f con todas las capacidades residuales positivas. Se puede aumentar f en $\Delta = \min c_f$ sobre el camino, avanzando en aristas forward y disminuyendo en reversas.

4. Teorema Max-Flow/Min-Cut (MFMC)

Las sig. afirmaciones son equivalentes:

- f es un flujo máximo.
- No hay caminos aumentantes en G_f .
- Existe un corte mínimo (S, T) tal y $|f| = \text{cap}(S, T)$.

Idea de demostración: (i) Si no hay aumentantes, ningún flujo puede crecer $\Rightarrow$ óptimo. (ii) La saturación de un corte $\Rightarrow$ se puede alcanzar de no alcanzables en G_f prueba $|f| = \text{cap}(S, T)$.

5. Algoritmos de flujo máximo.

5.1. Ford - Fulkerson (FF)

Idea: repetir: encontrar un camino aumentante y aumentarlo.

Pseudocódigo (FF):

Dejar abierto como
mantener el camino.

while exista camino P de s a t en el residual G_f :

$\Delta \leftarrow \min \{C_f(e) : e \in P\}$

para cada arista e en P :

si e es forward: $f(e) \leftarrow f(e) + \Delta$

si e es backward: $f(e^op) \leftarrow f(e^op) - \Delta$

Complejidad: depende de como se elige el camino. Con capacidades enteras termina y realiza a lo sumo $|V|^2$ aumentos. Si cada $\Delta \geq 1$. No es acotado polinomialmente sin especificar la regla.

Nota de implementación: usar BFS/DFS para encontrar caminos; mantener lista de adyacencia residual; cuidado al actualizar aristas reversas.

5.2. Edmonds - Karp (EK)

Regla: elegir siempre un camino más corto en número de aristas (BFS) en G_f .

• Complejidad: $O(VE^2)$. Cada arista se vuelve crítica $O(V)$ veces, y cada BFS cuesta $O(E)$.

• Ventajas: simple, robusto, suficiente para la mayoría de los ejercicios de la guía y personal.

Pseudocódigo (EK):

while BFS en G_f encuentre camino P de s a t :

$\Delta \leftarrow \text{cuello de botella}(P)$

aumentar f en P por Δ

5.3. Dinic.

Idea: construir un nivel graph por BFS (cappas de distancia desde s) y, sobre el, en ∞ flujo hasta obtener un blocking flow mediante DFS.

- Complejidad: $O(V^2E)$ en general. Muy rápida en la práctica, especialmente con capacidades unitarias.

Esguena:

maxflow $\leftarrow 0$

while BFS construya niveles (t alcanzable):

longitud iteradores de adyacencia

while exista camino bloqueante (DFS sobre niveles):

enviar en lo posible y acumular en maxflow

5.4. Scaling (aproximado)

Usar umbrales Δ (potencias de 2) y solo aumentar caminos con capacidad residual $\geq \Delta$. Mejora en redes con capacidades grandes:

$O(E^2 \log U)$, donde $U = \max e(u, v)$.

6. Transformaciones y modelado (claves en la práctica).

6.1. Varias fuentes / sumideros

- Crear super-source s' con aristas $s' \rightarrow s_i$ (capacidad adecuada).
- Crear super-sink t' con aristas $t_j \rightarrow t'$.
- Correr max-flow entre s' y t' .

6.2. Capacidades en vertices

- Reemplazar cada v por dos nodos v_{in} y v_{out} , con arista $v_{in} \rightarrow v_{out}$ de capacidad igual a la capacidad del vertex.
- Todas las aristas entrantes van a v_{in} ; todas las salientes salen de v_{out} .

6.3. Caminos disjuntos (Menger).

- Por aristas: poner capacidad 1 en aristas; max-flow = número máximo de caminos disjuntos por aristas.
- Por vertices: hacer split de vertices con capacidad 1 (excepto s, t).

6.4. Matching bipartito (emparejamiento máximo)

• Grafo bipartito $G = (L \cup R, E)$.

• Construcción de flujo: $s \rightarrow L$ (cap 1), aristas $L \rightarrow R$ (cap 1), $R \rightarrow t$ (cap 1).

• Max-flow = tamaño del matching máximo.

• Teorema: en bipartitos, tamaño de matching máximo = tamaño de vertex cover mínimo (se puede obtener del min-cut).

6.5. Asignación de tareas con cupos

• Si un elemento de L o R puede emparejarse con más de uno, usar capacidades > 1 en $s \rightarrow l$ o $r \rightarrow t$.

• Si hay incompatibilidades, simplemente no crear la arista $l \rightarrow r$ correspondiente.

6.6. Cotas inferiores y circulaciones con demandas (muy usado).

Objetivo: manejar restricciones $l(u,v) \leq f(u,v) \leq c(u,v)$ y la demanda de flujo en nodos.

Paso 1 - Remover cotas inferiores en aristas: para cada arista (u,v) con cota l , disminuir capacidad residual reducida $c'(u,v) = c(u,v) - l(u,v)$ y una carga de nodos: $b(u) += -l(u,v)$, $b(v) += +l(u,v)$.

Paso 2 - Balancear nodos (circulación factible): crear super-source S^* y super-sink T^* .

• para cada nodo con $b(u) > 0$ (necesita entrar flujo), agregar $S^* \rightarrow u$ con capacidad $b(u)$.

• para cada nodo con $b(u) < 0$ (debe salir), agregar $u \rightarrow T^*$ con capacidad $-b(u)$.

Correr max-flow de S^* a T^* . Si el flujo máximo satiene todos los nodos, desde S^* , existe una circulación factible y respeta cotas inferiores.

Paso 3 - Volver al problema original: sumar $l(u,v)$ a las aristas usadas en la solución reducida para construir el flujo real.

Caso $s \rightarrow t$ con cota $s \rightarrow t = 0$ con demanda en nodos: se usa la misma idea, pero conectando $t \rightarrow s$ con capacidad infinita (o grande) para transformar en circulación, o bien fijando balance de nodos y luego maximizando lo pedido.

6.7. Obtener el min-cut desde el residual.

Tras un max-flow, hacer BFS/DFS desde s en G_f . Los vertices alcanzables forman S , el resto T .
Las aristas $S \rightarrow T$ (originales) q estén saturadas conforman el corte mínimo.

7. Ejemplos comunes (mínimo).

7.1. Matching bipartito básico.

- $L = \{a, b\}$, $R = \{x, y\}$, aristas: $a \rightarrow x$, $a \rightarrow y$, $b \rightarrow y$.
- Construcciones con capacidades 1.
- Max-flow = 2 ($a \rightarrow x$, $b \rightarrow y$). Min-cut = 2.

7.2. Capacidades en vertices

- Nodos A, B, C con capacidades B igual a 2.
- Reemplazamos B por $B_{in} \rightarrow B_{out}$ con cap 2, reds orig. aristas.

7.3. Costos diferentes.

- Arista $u \rightarrow v$ con $l=2$, $c=5$.
- Poner $c_l=3$ y balancear: $b(u) += -2$, $b(v) += +2$.
- Agregar $S^* \rightarrow v$ cap 2, $u \rightarrow T^*$ cap 2, max-flow de S^* a T^* con pesos factibilidad.

8. Pseudocódigos

8.1 Edmonds-Karp (límite algebraico).

función edmonds-karp(G, s, t):

Construir residual R a partir de G (agregar arista reversa con cap 0).

maxflow $\leftarrow 0$.

mientras (padre := BFS-contrer, camino(R, s, t)) existe:

$\Delta \leftarrow +\infty$.

$v \leftarrow t$

mientras $v \neq s$:

$u \leftarrow \text{padre}[v]$.

$\Delta \leftarrow \min(\Delta, \text{cap-}R(u, v))$.

$v \leftarrow u$.

$v \leftarrow s$

mientras $v \neq t$:

$u \leftarrow \text{padre}[v]$

$\text{cap-}R(u, v) \leftarrow \text{cap-}R(u, v) - \Delta$

$\text{cap-}R(v, u) \leftarrow \text{cap-}R(v, u) + \Delta$.

$v \leftarrow u$.

maxflow $\leftarrow \text{maxflow} + \Delta$.

devolver maxflow y R (para min-cut).

8.2. Dinac (esqueleto).

función $\text{dinac}(G, s, t)$:
construir residual R .
 $\text{maxflow} \leftarrow 0$.
mientras $\text{BFS_nuevos}(R, s, t)$:
 $\text{iter}(u) \leftarrow 0$ para todo u .
 mientras $(\Delta := \text{DFS_bloqueante}(R, s, t, +\infty)) > 0$:
 $\text{maxflow} \leftarrow \text{maxflow} + \Delta$.
devolver maxflow .

DFS_bloqueante: avanza solo a nodos con $\text{nivel}[u] = \text{nivel}[v] + 1$ y $\text{cap}(u, v) > 0$; retrocede cuando se agotan.

8.3. Circulaciones con cotas inferiores (procedimiento).

para cada arista (u, v) con $l(u, v)$ y $c(u, v)$:
 $c'(u, v) \leftarrow c(u, v) - l(u, v)$
 $b(u) \leftarrow b(u) - l(u, v)$
 $b(v) \leftarrow b(v) + l(u, v)$.
crear S^*, T^*
para cada u con $b(u) > 0$: agregar $(S^* \rightarrow u)$ con $\text{cap } b(u)$.
para cada v con $b(v) < 0$: agregar $(v \rightarrow T^*)$ con $\text{cap } -b(v)$.
Si $\text{maxflow}(S^* \rightarrow T^*) = \sum_{b > 0} b(u)$: existe circulación factible.
reconstruir $p(u, v) \leftarrow f'(u, v) + l(u, v)$.

9. Cuando usar cada algoritmo.

- Edmonds-Karp: estándar, fácil de codificar, suficiente para la mayoría de los ejercicios ($O(V E^2)$).
- Dinic: cuando E es grande o hay muchas capas naturales (bipartitos, unidireccional).
- FF + scaling: cuando las capacidades grandes y estructura simple.

10. Recuperar soluciones y min-cut.

- Matching: quedarse con aristas ($\rightarrow$ con flujo 1).
- Caminos disjuntos: reconstruir caminos apartes de flujo en red de capacidad.
- Min-cut: nodos alcanzables desde s en el residual $\Rightarrow S$. Las aristas originales saturadas de $S \rightarrow T$ componen el corte mínimo.

11. Errores típicos y cómo evitarlos.

1. Olvidar las aristas reversas en el residual.
2. Sumar flujo sin chequear cuello de botella (Δ).
3. Comparar modelo (matching vs. caminos disjuntos vs. flujo en vertical).
4. Cotas inferiores: no ajustar balances $b(u)$.
5. Min-cut mal leído: se obtiene residual, no del grafo original directamente.

12. Guía de modelado (plantilla mental)

1. ¿Cada unidad u fluye y representa? (persona, tarea, paquete, etc.).
2. ¿Dónde puede salir y dónde puede terminar? (s, t).
3. ¿Cuál es el límite por gránulos? (capacidad o cota).
4. ¿Hay compatibilidades? (carrilones, aristas).
5. ¿Hay requisitos mínimos? (cotas inferiores).
6. ¿Puede haber más de un flujo, factibilidad o mínima costelower?
7. ¿Cómo se le reparte el flujo?

13. Complejidad y cotas.

- Edmonds-Karp: $O(VE^2)$.
- Dinic: $O(V^2E)$ general, muy eficiente en redes unitarias / bipartitas.
- FFaster: a lo sumo 10^4 aumentos.
- Scaling: $O(E^2 \log U)$.

¿Cómo demostramos en Flujo? N = red de flujo.

Tenemos g para el s y solo 5 .

Un flujo válido en N de valor $F \Leftrightarrow$ Hay solución posible para el problema P modelado con la red N de valor F .